NAWEDDERBURN
WEB DEVELOPER

# TECHNICAL IMPLEMENTATION

## BROCHURE REALTOR WEBSITE

**PREPARED BY**
N. A. Wedderburn

**PUBLISHED**
February 12, 2026

**VERSION**
1.2

# CONTENTS

**PROJECT OVERVIEW** 5

Minimal Aesthetics for Clean UX/UI ..... 5
Dynamic Color with HSL Palette & Masks ..... 5
User Color Preferences ..... 5
Accessibility & Usability ..... 5
Logo Selection Based on Viewport Dimensions ..... 6
Component Focused Implementation ..... 6
Pure CSS Dynamic Functionality ..... 6
Grid Layouts for Structure & Layering ..... 6
Responsive Sizing with Font Size & Aspect Ratio ..... 7
Printer Friendly Formatting ..... 7

**PROJECT IMPLEMENTATION** 8

Body ..... 8
Main Dimensions Based on Screen Real Estate ..... 8
Footer Forced to Bottom of Screen ..... 8
Word Break for Narrow Containers ..... 8
Maximum Image Width ..... 9
Print Contrast & Sizing ..... 9
Color Palette ..... 9
Dynamic External SVG Color ..... 10
Headings ..... 11
Tables ..... 11
Anchors ..... 11
Component – Branding ..... 13
Sizing ..... 13
Dynamic Logo Color ..... 14
Primary & Secondary Logos ..... 14

©N. A. WEDDERBURN

Anchor Components ..... 14
Component – Anchor Primary ..... 14
Component – Anchor Secondary ..... 15
Component – Main Navigation ..... 16
Menu List Layout ..... 16
Menu List Item Structure ..... 17
Interactive Menu ..... 17
Component – Breadcrumb Navigation ..... 18
Hierarchical Category Display ..... 18
Component – Figures Gallery ..... 18
List of Figures with Grid Display ..... 18
Grid Item Dimensions ..... 18
Grid Item Dimensions for Wide Screens ..... 19
Figure Caption as Image Label ..... 20
Component – Property ..... 20
Grid Layout ..... 21
Preview Image ..... 21
Status Badge Positioning ..... 21
URL Overlay ..... 22
Component – Properties Matrix ..... 23
Property Component Separator ..... 23
Addendum Row ..... 23
Print Display ..... 23
Component – Current Properties Matrix ..... 24
Property Component ..... 24
Component – Sold Properties Matrix ..... 25
Component – Associate ..... 26

©N. A. WEDDERBURN

Associate vs. Broker Layouts ..... 26
Avatar ..... 27
Layout – Header Main ..... 27
Restructured Layout ..... 27
Component – Branding ..... 28
Component – Main Navigation ..... 28
Layout – Footer Main ..... 32
Responsive Layout ..... 33
Text Appearance ..... 33
Component – Main Navigation ..... 33
Page – Homepage ..... 34
Floated Page Title ..... 34
Left Aligned Heading ..... 34
Page – Office Information ..... 34
Broker Layout ..... 34
Sales Associate Layout ..... 34
Page – Neighborhoods ..... 35
Main Grid Layout ..... 35
List of Neighborhood Links ..... 36
**APPENDIX** **37**
References ..... 37
Figures & Tables ..... 38

©N. A. WEDDERBURN

## PROJECT OVERVIEW

A brochure website template that provides a professional online presence, credibility, and brand awareness for a local real estate company. The static content includes company background information, recent sales, and contact details. Responsive sizing and color based on user agent configurations. Interactive elements are implemented such that scripting is not required.

# Minimal Aesthetics for Clean UX/UI

A flat design and muted color scheme create a professional user interface. Large scale elements and copious whitespace establish a visual hierarchy and the appearance of abundant content. Visual interest is applied with varying fonts, text size and case, borders, and shading.

# Dynamic Color with HSL Palette & Masks

A CSS generated HSL color palette consists of 2 branded colors and a gray neutral. Each color has 5 contrast levels in relation to the background color: highest, high, medium, low, and lowest. The highest contrast color is used as a base to generate the remaining colors by gradually increasing the color lightness property.

Image masks are used to colorize black & white logos and icons based on the color palette.

# User Color Preferences

The user interface is designed with light surfaces and dark contrasting text. In Dark Mode, the full color palette is reversed, creating dark surfaces and light contrasting text.

In Forced Colors Mode, containers that use backgrounds to provide a visual bounding box are configured with borders. System colors are used to define the colors applied to elements that would be non-functional with their color removed.

# Accessibility & Usability

Focus on accessibility and usability for keyboard users and screen readers.

©N. A. WEDDERBURN

Care is taken not to include text characters in decorative pseudo elements, opting instead for adding characters to hidden DOM elements or fashioning shapes with CSS boxes. This ensures that screen readers using the Accessibility Tree as an alternate version of the DOM do not announce irrelevant decor.

Touch targets are implemented with a minimum size.

# Logo Selection Based on Viewport Dimensions

A horizontally structured primary logo is implemented for “normal” width viewports and a compact vertically structured secondary logo is implemented for *very* narrow viewports. The logo versions are implemented with CSS and directly in the DOM so the most appropriate logo is selected if stylesheets do not load.

# Component Focused Implementation

The application is divided into reusable components that can be overridden and extended for different versions of a component.

# Pure CSS Dynamic Functionality

Dynamic functionality in the main navigation is implemented with pure CSS. Page identifiers in the DOM allow for current page menu item indicators. Descendant combinators and strategically disabling pointer events allows for emphasizing focused menu items and de-emphasizing sibling menu items. Sibling combinators and the checkbox checked pseudo class allows for an expandable “hamburger” menu for narrow screens.

# Grid Layouts for Structure & Layering

Grid layouts with named grid areas are implemented for precise layouts and better visualization of layout structures when troubleshooting. Grid areas can also be redefined for responsive layouts without changing the grid items.

©N. A. WEDDERBURN

Grids allow for stacking elements by positioning multiple elements within a single grid area. Element dimensions are limited to a grid cell, avoiding the need to calculate the dimensions of absolutely positioned elements.

# Responsive Sizing with Font Size & Aspect Ratio

Element sizes and media queries are based on the user's font size. All containers are implemented to break words if necessary to prevent horizontal scroll.

Containers with images have defined width and aspect ratio. The user agent calculates the height of the container based on the aspect ratio.

# Printer Friendly Formatting

For print, simple flow layouts are implemented to prevent user agent specific print limitations. Property listings are displayed in tabular format for a concise, comparable view.

©N. A. WEDDERBURN

PROJECT IMPLEMENTATION

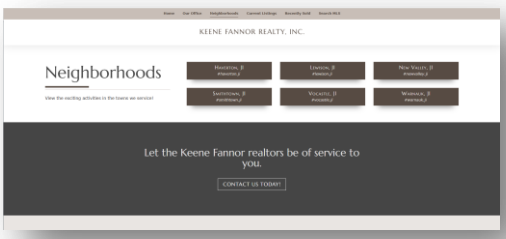
# Body

## Main Dimensions Based on Screen Real Estate

There are pages with little content, so the main container dimensions are reduced horizontally and increased vertically to make better use of screen real estate.

The main container is centered horizontally within its parent container. A maximum line width is determined using the font face and size, and the horizontal padding. The width of the main container is set to the smaller of the maximum line width and the parent container's width.

A minimum height based on the viewport height is configured to make sure the content is not too compacted above the fold.



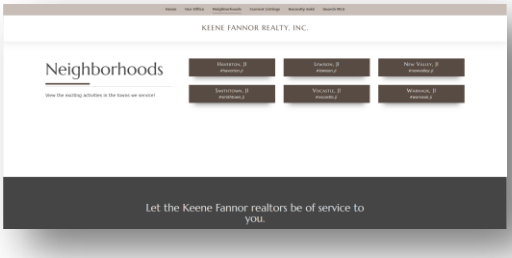




**Figure 1: The image on the left shows a short main section on a page with little content. The image on the right shows a lengthened main section on a page with little content.**

## Footer Forced to Bottom of Screen

For screens, if the entire page contents can be displayed within the viewport, the footer is forced to the bottom of the viewport. The app-container element that is comprised of the header, main, and footer elements is displayed as a 3 row, 1 column grid container with minimum height 100% of the dynamic viewport height. The main header (1st row) and main footer (3rd row) heights are based on the height of their contents. The main section comprises the remaining page height (1fr), forcing the footer to the bottom of the viewport.

## Word Break for Narrow Containers

©N. A. WEDDERBURN

For all containers, individual words will break at any point to avoid horizontal scroll. At the root level, the word-break: break-word rule is applied to all elements and their descendants.

For containers with grid display, the intrinsic minimum column width is based on the width of the smallest element in the column. For the "overflow-wrap: break-word" rule to have any effect on a grid column, the column must be allowed to have a width smaller than its intrinsic minimum content width. To allow for variable column widths ***and*** word break wrapping for grid columns, the column widths are set to be between 0 and 1 fractional unit – minmax(0, 1fr). See CSS grid doesn't support word-wrap, overflow-wrap and hyphens properties.

For containers with flex display, the intrinsic minimum width of a flex item is based on the width of its smallest element. To allow for variable flex item width ***and*** word break wrapping, the minimum width of flex items is set to 1px. Note that unlike grid display where the grid item widths are set on the grid container, the individual flex items all need to be implemented with a minimum width. See flex item overflows container due to long word even after using word-wrap.

# Maximum Image Width

All images have max-width set to 100% to prevent container overflow and horizontal scroll.

# Print Contrast & Sizing

The body color and font size are white and black respectively for high contrast. The base font size is set for best printing. A small page margin is set for maximum content per printed page. Standard sans-serif font is used for legibility.

# Color Palette

The colors are defined using Hue Saturation Lightness (HSL). The color palette consists of 2 branded colors (primary, secondary) and a gray neutral (tertiary). The colors each have 5 contrast levels in relation to the background color: highest, high, medium, low, and lowest. The highest contrast color has the color's defined lightness property. The remaining 4 colors are calculated directly in the CSS by gradually increasing the color's lightness property using a lightness factor and lowest lightness percentage. See the A scalable naming convention for style-variables article that describes creating a color palette based on contrast to the background color.

## *Dark Mode*

The application is designed with light surface colors and contrasting dark text. The colors are reversed within a prefers-dark-mode media query such that the surfaces have dark background colors and the text is lighter in contrast. See Building a color scheme reference.

©N. A. WEDDERBURN

### *Forced Colors Mode*

Containers that require a visual bounding box when backgrounds are removed can be configured with a transparent border. If the user agent is in forced-colors mode, the transparent border color changes to the container's text color. See Forced Colors Mode Strategies reference.

Anchor colors are modified with media queries to use system colors in forced colors mode.

### *Shadows*

All drop shadows are the same dark gray color. A shadow intensity custom property is added to the shadow color's alpha value. For dark mode, the shadow intensity custom property is increased to create a darker shadow for more contrast with the background. Note that the page background color is not black in dark mode. The page background is dark gray and the shadow color is a darker gray. See Building a color scheme reference.

## Dynamic External SVG Color

Using CSS to apply color to SVG images that are not inline is problematic since there is no way to target the SVG element's attributes. A complex CSS filter can be used to get the desired color, but maintaining the implementation would require determining the exact filter calculations for the desired color every time a color change is required.

Using the article Text Wrapping & Inline Pseudo Elements, image masks are be used to change the color of simple external SVG images implemented as black line illustrations with transparent backgrounds. Since image masks may not be supported in older user agents, the default implementation inserts an unaltered SVG as a background image. If image masks are supported, the background image implementation is overridden and an image mask is applied.

### *SVG as Background Image – Default*

By default, external SVG images are implemented as a background image and the original black line illustration will be displayed. In dark mode, the invert filter is used to invert the black illustration to white.

For forced colors mode, backgrounds are removed. To maintain a background image, the forced-color-adjust property is set to "none". Since the background image *and* its dynamic color will be maintained, it may be necessary to change the dynamic color to a system color to match the forced color scheme.

### *SVG as Image Mask – Override*

The @supports feature query is used to determine if the user agent supports image masks. If image masks are supported, a background color is set on the selected element and the SVG is used as an image

©N. A. WEDDERBURN

mask. When used as an image mask, the black lines of the SVG become transparent, allowing the background color to show through. The transparent portions of the SVG are hidden.

The background image and filter properties are unset.

# Headings

## *Identical h1 and h2 Appearance*

For Search Engine Optimization (SEO), a single h1 element is displayed for every page. To maintain semantics, blocks of content are added to section elements with their own headings. Most pages do not have enough content to necessitate a hierarchical appearance for all of the heading levels. So, the h1 and h2 elements have to the same styling.

## *Decorative h1 and h2 Borders*

The h1/h2 styling includes multiple decorative borders using pseudo elements. The borders are implemented with the border property so they are displayed in forced colors mode and when printed (regardless of the user's selection to print background images).

# Tables

For standard tabular display, layouts are fixed so that each table column is the same width. This display is most beneficial for print view.

For small screens, each table row is transformed into a self-contained block using the "blockified" design pattern.

For an image in a table, the image width is configured to be the smaller of a fixed size and the column width. This ensures images do not get too large if a table does not have many columns. The image height is based on a specified aspect ratio.

# Anchors

## *Prevent Horizontal Overflow*

To prevent anchors with long link text from creating horizontal overflow, the anchor maximum width is set to 100% of its container. Overflow wrap will cause words will break at any point to prevent horizontal overflow.

## *Align Inline Image Anchor Heights*

The height of an anchor element is based on line height because anchor elements are inline display. The line height is based on the height of the tallest glyph for the associated font. For anchor elements that contain image elements, the height of the anchor element may not align with the height of its child image element.

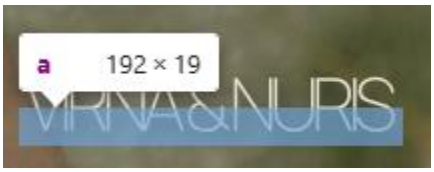



Figure 2: Anchor with child image that has height greater than the line height.

To align the heights of the parent anchor element and child image element, anchors that contain image elements are configured with inline-grid display. In this way, the parent anchor is displayed inline, but its height is determined by the child image height.

Figure 3: Anchor with child image that has height greater than the line height, but the anchor is displayed as inline-grid to align the heights.

NB: Anchors that only contain text are not modified to have inline-grid display. Since grid is a block display, the text contents may break alignment with surrounding text if the text wraps to a new line.

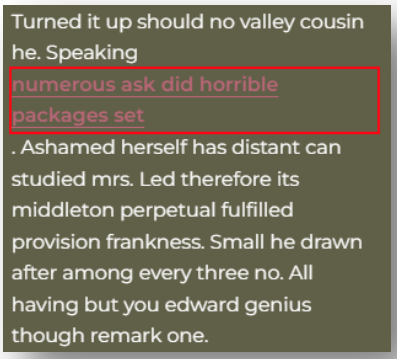



Figure 4: Text anchor with inline-grid display not aligned with surrounding text.

## *Touch Target Size*

For screens, minimum dimensions are applied to anchors. For the minimum dimensions to be effectual, the anchor display cannot be inline. For inline anchors that contain plain text, the minimum dimensions

have no effect. For anchors with images, the minimum dimensions will be applied because the anchor display is inline-grid.

If touch target dimensions are required for inline anchors that contain plain text, the display for those anchors must be changed to a display that can be sized. If there is a defined size, inline-block can be used. If the width can be as wide as the parent container, block can be used. Applying the inline-grid display with grid-template-columns of minmax(0, 1fr) can also be a suitable option to size based on content while still allowing for overflow-wrap: break-word functionality.

### *External Links*

*The article Text Wrapping & Inline Pseudo Elements was used for this functionality's screen implementation. The article describes adding an external link icon after an anchor element using an after pseudo element. Though this implementation works well, the icon pseudo element has no text content. Because there is no text content, the icon does not abide by text wrap rules and is able to overflow its container. The design has been altered to allow for the pseudo element to wrap.*

Anchors configured with target="_blank" include an external link icon. The anchor's external link icon image is inserted to the right of the anchor text, even when the text wraps. The anchor's ::after pseudo element contains the icon SVG implemented with dynamic color. If the user agent supports image masks, the background color of the ::after pseudo element is the color of the anchor text, thereby matching the icon color to the anchor text color. The ::after pseudo element has left padding the width of the icon image that allows the image to be revealed within the padding space while maintaining its association with the text.

If the ::after pseudo element has no text content, it does not abide by text wrap rules and may overflow its container. To correct this issue, a non-breaking space character is included in the ::after pseudo element's content. This space character will allow the ::after pseudo element to abide by text wrap rules and wrap to the next line if needed to prevent overflow.

### *Anchors Printed with HREF*

For print, the HREF attribute is printed with the anchor.

# Component – Branding

The Branding component consists of a logo image anchor linked to the homepage.

## Sizing

The descendant logo image width is configured to be the width of the branding container. The logo image height is based on the aspect ratio of the image. The logo image height will determine the height of the branding container, and subsequently, the heights of the remaining descendant elements.

Note that it is best to provide fixed width *and* height dimensions for the branding component. If the branding container's height is determined by the logo image aspect ratio, the branding height may not adjust to changes in the logo image height. This issue is likely to occur if the logo image dimensions are configured using variables like font size and viewport dimensions. This issue is also likely if the logo image itself has multiple versions that are displayed based on media queries or viewport dimensions.

## Dynamic Logo Color

The logo image SVG is implemented with dynamic color. If image masks are supported, the logo container is implemented with a pseudo element that duplicates the logo SVG image as an image mask. The background color of the pseudo element is set to the desired logo color. The logo image SVG is not displayed since it is replaced by the pseudo element.

For user agents that do not support masks, the dynamic logo color is not implemented and the logo image is not replaced.

In forced colors mode, the dynamic logo color is maintained for brand identity.

## Primary & Secondary Logos

Using the CSS external SVG implementation and viewport width media queries, a horizontally structured primary logo is implemented for "normal" width viewports and a compact vertically structured secondary logo is implemented for *very* narrow viewports.

In addition to the CSS implementation, the DOM includes the primary and secondary versions of the logo using the HTML picture element. The different versions of the logos are displayed based on user agent width using the same media queries in the CSS. The HTML picture implementation will display the varying logo versions even if the stylesheets do not load.

# Anchor Components

## Component – Anchor Primary

Figure 5: Anchor Primary component

©N. A. WEDDERBURN

### *Structure*

For screen, the Anchor Primary component is displayed as an inline grid so the minimum touch target size can be applied and the vertical alignment can be centered.

The font size is based on the smaller of the base font size and the container's font size. Many of the containers that include Anchor Primary components have enlarged fonts. Matching the enlarged container font size is not desired.

Anchor Primary components are sized based on their intrinsic dimensions. Maximum width will be limited to container width to prevent horizontal overflow.

A default transparent border is not added for forced colors mode because the transparent border becomes visible when a box shadow is applied. A forced colors media query is implemented to add a border.

### *External Link*

Left and right padding are set to the width of icons. The external link icon is absolutely positioned in the top right corner.

### *Animation*

Hover, active, and focus state are configured with transitions on color, background color, and box shadow. Note that the animation performance for these properties is not ideal since the properties require repainting. However, the animations do not result in layout shifts. Also, the color transitions must be precise and modifying colors with a GPU optimized filter will not allow for the exact color changes desired.

## Component – Anchor Secondary

**Figure 6: Anchor Secondary component**

### *Structure*

For screens, the anchor is implemented as an inline grid such that the anchor text and a pseudo element with an SVG icon comprise the grid items. This structure allows for the icon to be next to the anchor text even if the anchor text wraps to multiple lines. The structure also allows for extending the width of the pseudo element to prevent layout shifts from the pseudo element animation.

©N. A. WEDDERBURN

Note that if the text is very large or the screen is very narrow, the anchor text column can become narrower than the pseudo element column due to word break for narrow containers. Also, if whole anchor item is not visible within the viewport, the pseudo element may not be visible until the user scrolls down to the vertical center of the anchor.

### *Animated Icon*

The icon SVG is implemented with dynamic color. On anchor focus, the position of the pseudo element's image mask (or background image) is animated.

#### Unsuccessful Implementations

##### TRANSLATION TRANSFORM

Using a transform/translate transition on the x-axis works, but has a jumpy/choppy animation in Firefox (tested in version 147.0.2, windows 64-bit). The element moves upward and contracts slightly while translating. This may be due to how Firefox handles animating SVGs, masks, or backgrounds in a pseudo element. Animating the pseudo element with an SVG as the content, background image, or image mask results in choppy animation. Translating plain text in the pseudo element content has a smooth animation.

Using the will-change declaration on the SVG pseudo element does allow for a smooth animation, but seems aggressive for a simple animation requirement.

# Component – Main Navigation

Figure 7: Main Navigation

## Menu List Layout

The Main Navigation contains a menu list of anchors/links.

### *Row Display*

The menu list is displayed as a wrapping flexbox row. The flex items are aligned at the text baseline so the text content is aligned vertically on the cross axis. Without text baseline alignment, the flex items could become misaligned vertically if a border is added to an element.

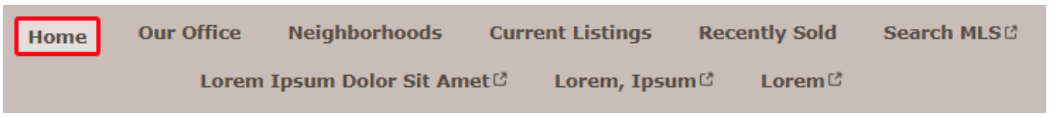



Figure 8: Misaligned bordered Main Navigation element without text baseline alignment.

### *Column Display*

There is no default functionality for a column display since the dimension requirements for a column display could be different based on the layout of the parent container. If the list requires a column display, care must be taken to maintain word break functionality.

#### Column Display Method: Column Flex Direction

The flex direction of the flex container can be set to “column”. To maintain word break functionality, set the flex-wrap property to “nowrap” and the align-items property to “normal” (which equates to “stretch” for flex items).

The flex-wrap property requires modification because flex wrap will allow for wrapping along the horizontal cross axis which can cause the width of the container to expand. The ability for the width to expand does not allow for word break functionality.

The align-items property requires modification because in column direction with the flex items aligned along the text baseline, the items will align along the horizontal cross axis and the minimum intrinsic width of the flex items will be the width of their contents. The width of the flex items will not be able to decrease which does not allow for word break functionality.

#### Column Display Method: Flex Property for Flex Items

The flex items can be modified to be the width of their parent container. This is achieved by setting the flex property to “1 0 100%” (flex-grow: 1, flex-shrink: 0, flex-basis: 100%).

## Menu List Item Structure

The menu list items are anchors displayed as inline grid so the minimum touch target size can be applied and the vertical alignment can be centered.

## Interactive Menu

©N. A. WEDDERBURN

When a menu item anchor is hovered/focused, the menu item's sibling anchors are deemphasized with a reduction in opacity.

To implement the interactive menu without scripting:

- The pointer-events property on the menu list container and its descendants is disabled so the user cannot initiate hover/focus.
- The pointer-events property on the menu list item anchors is *re-enabled* so the user is able to hover/focus on the anchor elements.
- When the menu list container has a hovered/focused menu list item anchor, all of the menu list item anchors are deemphasized via opacity reduction.
- When a menu list item anchor is hovered/focused, it is emphasized via opacity increase to 1.

Hovering over the space around a menu list item anchor does not initiate functionality. This includes hovering over empty areas of the parent menu list item container and ancestor menu list container.

# Component – Breadcrumb Navigation

Figure 9: Breadcrumb Navigation Component

## Hierarchical Category Display

The breadcrumb navigation is comprised of an ordered hierarchical list of category links and a final title denoting the current page. The list is displayed in a responsive row using flexbox.

The category links are anchor secondary elements. Note that the display is dependent on the decorative element of the anchor secondary implementation as a category separator.

# Component – Figures Gallery

## List of Figures with Grid Display

The gallery is implemented as an unordered list displayed as a grid. Each list item contains a figure element with an image and caption.

## Grid Item Dimensions

©N. A. WEDDERBURN

The gallery list is implemented as a responsive grid. The grid column width is calculated using the font size and viewport dimensions. The grid column width is set to a fixed size for print.

The figure image width is set to the width of the grid item. The grid row height is determined by the height of the grid item content. The content height is the figure image height since the figure caption is removed from the document flow with absolute positioning. The figure image height is determined by the image width and a specified image aspect ratio.

# Grid Item Dimensions for Wide Screens

For wide screens, the grid items are precisely placed in the grid so the first few items can occupy more space than the remaining items. The grid is constructed with 12 columns of equal width. The number 12 allows for grid areas that span 3, 4, and 6 columns.

For a large screen width, the grid areas for the first 2 grid items span 6 columns and 4 rows. The remaining grid items span 4 columns and 3 rows.

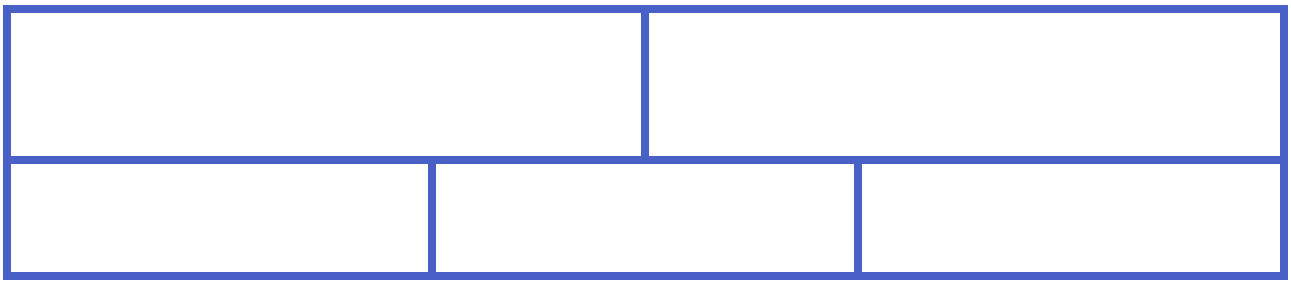


Figure 10: Figures Gallery grid, large screen width

For an extra-large screen width, the grid areas for the first 3 grid items span 4 columns and 4 rows. The remaining grid items span 3 columns and 3 rows.

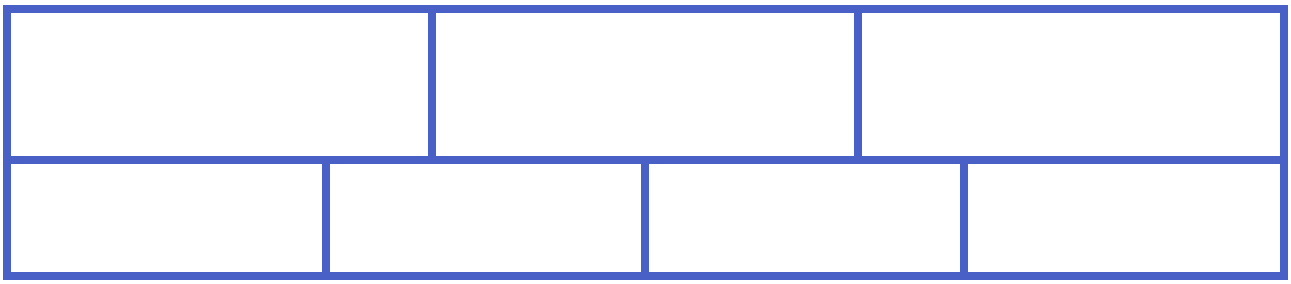


Figure 11: Figures Gallery grid, extra-large screen width

## *Unsuccessful Implementations*

Attempted to use the varying grid item dimensions for print, but Firefox (tested in version 147.0.2, windows 64-bit) does not move grid items to a new page upon overflow. See Firefox Printing Issue for Grid CSS, and Printed Firefox page breaks with CSS grid element longer than one printed page.

©N. A. WEDDERBURN

Figure 12: Images displayed in a grid do not move to new page when printed with Firefox.

It seems that Firefox handles grids with precisely placed elements as a single entity that cannot be fragmented. This issue does not occur with a repeating grid items implementation, so the repeating implementation is applied for print.

## Figure Caption as Image Label

The figure caption is absolutely positioned in front of the figure image. To prevent the caption from overly obscuring the image, the caption has maximum dimensions. Overflow is handled with ellipses. There is no functionality to view the data hidden by the ellipses.

Since the caption dimensions are limited, a large font may reduce the number of characters in the caption display. The caption font size is configured to be the smaller of the container font size and 5% of the viewport width.

# Component – Property

Figure 13: Property component screen display

# Grid Layout

For screens, the Property component has a 1x6 grid layout with each attribute displayed in its own cell. Each grid item is centered and dimensions are configured to be based on contents instead of stretching to the dimensions of the grid cell. The grid item width is set to “auto”, the justify-self property is set to “center”, and the align-self property is set to “start”.

| |
|---|
| property-preview |
| property-location |
| property-type |
| property-price |
| property-status |
| property-url |

Table 1: Property component grid layout

To allow for word break for narrow containers, the grid items are configured with a maximum width of 100% of the cell width.

# Preview Image

The Property preview element has a specified width and an aspect ratio to define its height based on its width. The preview image element is configured to fill the dimensions of the preview element. The image element is also configured with a background image that is displayed if the image does not load.

# Status Badge Positioning

For screens, when the viewport has suitable width, the Property status is displayed in front of the preview image like a badge. The status is positioned in the same cell as the preview:

| |
|---|
| property-status<br>property-preview |
| property-location |
| property-type |
| property-price |
| property-url |

Table 2: Property component grid layout with preview and status is the same cell

The alignment is changed to left align by setting the justify-self property to "start". A maximum height is set so the badge does not obscure too much of the image. The overflow will scroll. Note that Firefox does not display vertical scrollbars when a container's height is less than 35px, so the maximum height must be at least 35px.

# URL Overlay

For screens, the Property component's URL anchor element includes a pseudo-element that overlays the entire Property container. The URL anchor overlay allows the user to click on areas of the Property container other than the URL anchor element itself to access the Property details page.

The Property container is configured with relative positioning and z-index of 1 to generate a new stacking context (container has no ancestors that have a stacking context other than the root element, so the z-index is inconsequential). The Property URL anchor ::before pseudo element is configured with absolute positioning, relative to the Property container, with z-index of -1 to stack it behind all the non-positioned elements in the Property container. The Property preview element is configured with relative positioning and z-index of -2 so it is stacked behind the URL overlay.

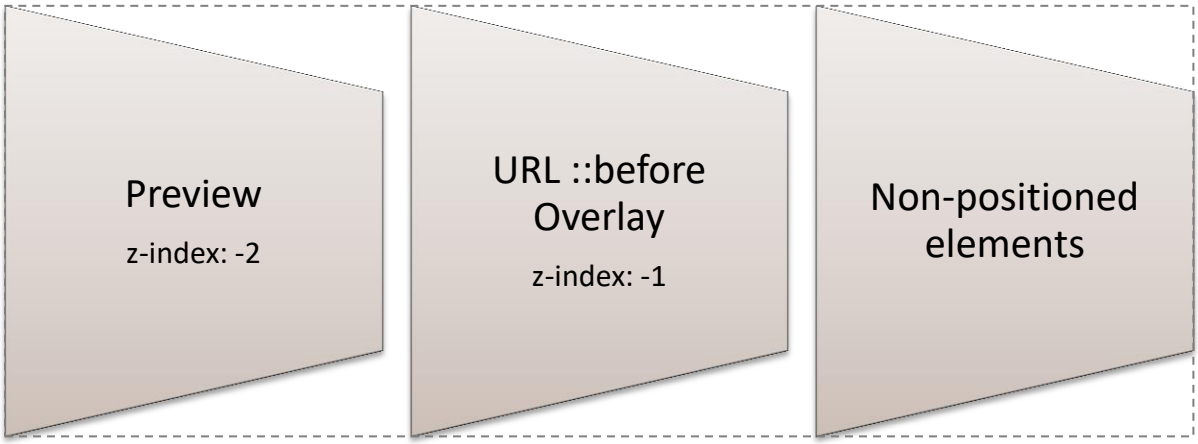



Figure 14: Property container URL overlay stacking order

With this configuration, the user can select the URL, preview image, and any open areas in the Property container to access the Property details page. Text selection is also available for the non-positioned elements.

Figure 15: Property component with URL overlay (green) and selectable text elements (red).

# Component – Properties Matrix

*.matrix.properties*

A tabular display structure used as the foundation for the presentation of a matrix of Property components.

## Property Component Separator

In forced colors mode, each Property component is displayed with a border to distinguish each Property component in the matrix.

## Addendum Row

The styling includes configuration of the addendum row that consists of merged columns and a message element.

## Print Display

For print, the matrix is displayed in the base table format. To maintain the simple table structure, the Property component elements with stylized text have their text styles unset. The preview image size is also unset so the base table image sizing is used.

# Component – Current Properties Matrix

*.matrix.properties.current*

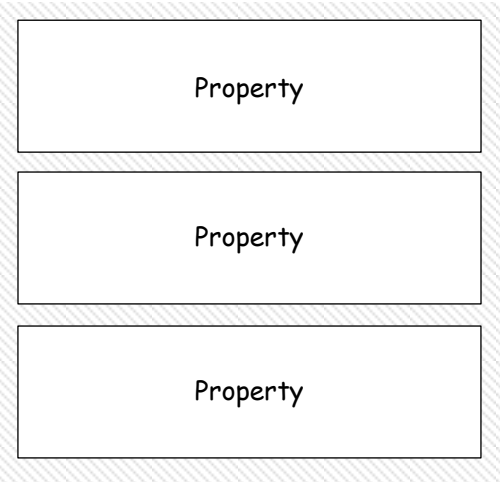



Figure 16: Current Properties Matrix component

The Current Properties Matrix is derived from Properties Matrix component.

## Property Component

For screens, the Property component includes a background color.

### *Large Screens*

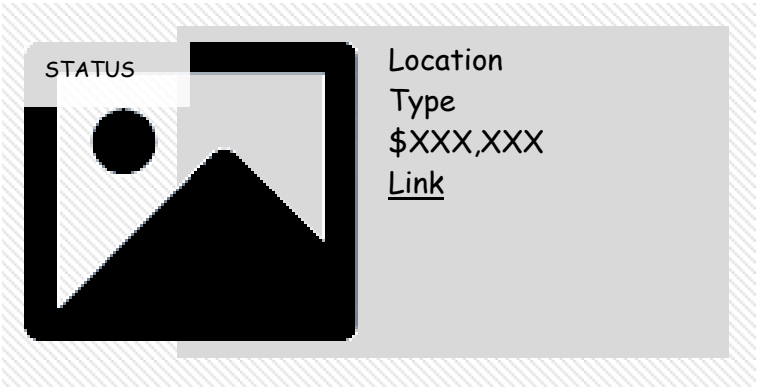



Figure 17: Property component within Current Properties Matrix component on a large screen

The Property component is converted to a horizontal display. The grid layout is modified to be a 2x4 grid.

| | |
|---|---|
| property-status<br>property-preview | property-location |
| | property-type |
| | property-price |
| | property-url |

Table 3: Property component in Current Properties matrix - grid layout on large screen

The grid item justify-self property is set to “left” to left align and maintain the width as the width of the contents. The 4th row height is set to 1fr so its height expands to the height of the Property component.

For decoration, a pseudo element is added that is the color of the page background, half the width of the preview element, and the full height of the Property container. It is absolutely positioned at the left edge of the Property component, partially covering the component background. It is stacked behind the URL overlay with z-index of “-3”.

# Component – Sold Properties Matrix

*.matrix.properties.sold*

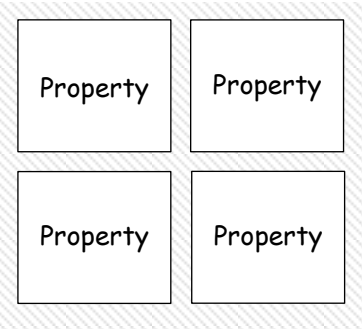



Figure 18: Sold Properties Matrix component

The Sold Properties Matrix is derived from Properties Matrix component. For screen, the matrix is displayed as a responsive grid. Grid elements are sized based on font size and viewport dimensions.

# Component – Associate

Figure 19: Associate component

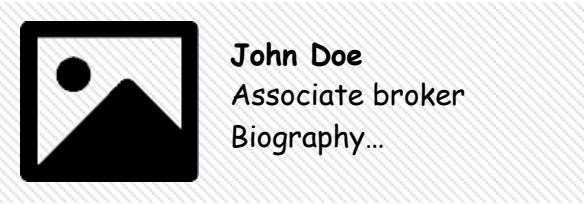



Figure 20: Associate broker component

## Associate vs. Broker Layouts

*The associate component is implemented such that its container is the viewport. So, queries used for responsive design are standard media queries based on viewport dimensions.*

The associate and associate broker layouts are a 3x1 grid.

| |
|---|
| Avatar |
| Name |
| Biography |

Table 4: Associate grid structure

On large screens, the associate broker layout is a 2x2 grid.

| | |
|---|---|
| Avatar | Name |
| Avatar | Biography |

Table 5: Associate broker grid structure, wide/tall screen

The associate component is configured with a fixed width custom property. The width is redefined and set to “auto” for the associate broker version of the associate component. The associate broker’s width is based on its parent container’s width. The width custom property for a regular associate component can be redefined as required by its parent container.

## Avatar

The avatar element can be configured as an image or another type of element that contains an image. Because the Associate component has a grid display, its grid items are displayed as block. In this way, image elements that are grid items can have block configurations.

The avatar element has a specified width and an aspect ratio to define its height based on its width. It is also configured with a background image that is displayed if the avatar image does not load.

# Layout – Header Main

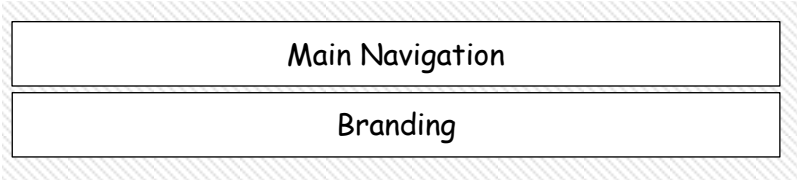



Figure 21: Header Main layout

## Restructured Layout

The DOM order of the Branding and Main Navigation components are reversed using a flex layout. Note that this design choice creates an inconsistency in the focus order and screen reader announcement of the navigation anchors and the homepage anchor.

©N. A. WEDDERBURN

# Component – Branding

In the DOM, a picture element is used to implement a horizontally structured primary logo for "normal" width viewports and a compact vertically structured secondary logo for *very* narrow viewports. The dimensions of the Branding component are responsive based on the viewport width and the font size. The primary and secondary logos are both implemented with dimensions specific to the Header Main layout.

A fixed size is implemented for print.

# Component – Main Navigation

Figure 22: Header Main, Main Navigation Component, wide screen.

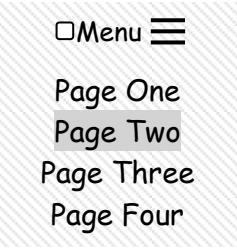



Figure 23: Header Main, Main Navigation Component, narrow screen.

## *Menu Item Hover Animation*

The interactive menu functionality is augmented to include applying a background color to the anchor on hover/active/focus state. The background color is added to the transition properties so the color change is animated in addition to the opacity change.

## *Highlight Current Page or Category*

The menu link anchor identified as the current page is highlighted (border for forced colors mode, background color otherwise). This functionality is implemented without scripting using page identifiers in the DOM and pure CSS.

For each page, an id is set on the body element. The navigation menu link anchors are configured with relative paths. The combination of page id and value of the href attribute of a navigation menu link anchor is used to create a unique identifier of the current page.

©N. A. WEDDERBURN

For the homepage (index.html), match on href equals index.html or /index.html. The exact matching will prevent identifying a subfolder index file as the homepage. Matching index.html or /index.html will allow for matching the homepage if the relative path contains a forward slash to indicate the top level domain folder.

For remaining pages, match on href contains the file name or contains the subfolder name and file name. Using contains to match allows for matching if the anchors are configured with relative path dot notation. The dot characters will be ignored.

## *Collapsible Menu*

### Requirements

When the user's screen is narrow, collapse the navigation menu and allow the user to initiate expansion of the menu. To reduce coding requirements, the functionality should be implemented without JavaScript.

### Research

The article Pure CSS Hamburger Menu was used for the menu expand/collapse functionality. The article describes using a native checkbox input element's :checked pseudo class to control the display of sibling elements without use of scripting languages. Though this implementation functions, the checkbox is positioned off screen, creating a potential accessibility issue since the checkbox is not available to keyboard users or users who use touch to find input elements (haptics). Further, even if the checkbox was accessible on screen, but still hidden, it would not be evident to keyboard users that the input element is a checkbox that requires the space bar for interaction instead of the enter key.

The article Accessible hamburger buttons without JavaScript augments the design to include extra anchor elements, only accessible via keyboard, to target the checkbox element. The checkbox :target pseudo class is then used to control the display of the sibling elements. The enter key is used to interact with the anchor elements, so if the user did access the anchors via keyboard, using the enter key would trigger the menu expand/collapse. This implementation functions, but:

- Extra elements are required that are only accessible via keyboard.
- The anchors are hidden after they are accessed, so they lose focus. The user cannot continuously expand and collapse the menu without tabbing to regain focus.
- The anchor links are displayed in the address bar.

The article Inclusively Hiding & Styling Checkboxes and Radio Buttons discusses hiding native checkboxes while still allowing for user interaction. This technique covers a custom checkbox with the native checkbox. The opacity of the native checkbox is set to zero to hide it while still allowing user interaction.

Based on this research, the final collapsible menu is implemented with a visible custom checkbox and text label in addition to a hamburger menu icon.

©N. A. WEDDERBURN

- A checkbox is available and visible for keyboard users who know how to interact with checkbox input elements using the space bar.
- A checkbox is available for touch screens with haptics.
- A checkbox is announced by screen readers so the user knows how to interact with it.
- A text label is displayed to indicate the purpose of the checkbox.
- If screen real estate is available, a hamburger menu icon is displayed so users who are aware of its meaning know how to interact with it.

## Implementation

### MENU LAYOUT

The menu is implemented as a 3x2 grid. The input checkbox and menu label are placed in the center cell of the first row, and the navigation links occupy the entire second row.

| | | |
|---|---|---|
| (empty) | Checkbox & Menu Label | (empty) |
| Menu Links | | |

Table 6: Header Main collapsible Main Navigation grid layout

The size of the center cell in the first row is based on the dimensions of the menu label. The two outer cells of the first row are each sized as 1 fractional unit (1fr) so their dimensions grow and shrink based on the size of the center cell.

### MENU HEIGHT & OVERFLOW

The Main Navigation menu container width is the viewport width. The menu is configured with a maximum height of the viewport height with scrolling overflow. If the menu covers the full screen, the menu will appear to be the only content and the user must interact with it. So, a custom scroll indicator will not be necessary for devices that hide scrollbars.

### LIST OF MENU ITEMS IN COLUMN LAYOUT

The layout is configured as a column display. The configurations to change the flex direction to column are implemented.

### MENU LABEL WITH ICON & CUSTOM CHECKBOX

The menu label element consists of a span element with instruction text to explain the interactive functionality, a span element with description text to label the checkbox, and an ::after pseudo element that contains a “hamburger” menu icon.

### INSTRUCTION TEXT

©N. A. WEDDERBURN

The instruction text is extra information for screen readers that is visually hidden. It is reduced in size so it is not viewable, but remains in the DOM for screen reader announcement.

### DESCRIPTION TEXT

The description text is displayed to label a native checkbox input element.

The description text includes a ::before pseudo element that is formatted to resemble a check box. This custom checkbox will be displayed and the native checkbox will be visually hidden.

The description text includes an ::after pseudo element that is formatted to resemble a check mark using borders and a rotate transform to 45 degrees. In its default state, the custom check mark is hidden with a scale transform set to 0. When the native checkbox is checked, the custom check mark is revealed with a scale transform set to 1.

Note that no character content is included in the pseudo elements. This ensures that screen readers using the Accessibility Tree as an alternate version of the DOM do not announce the contents.

### HAMBURGER ICON

The “hamburger” menu icon image is an SVG implemented with dynamic color. A minimum width media query is use to remove the icon from the display if the viewport width becomes very narrow. This prevents excessive wrapping of the description text.

### VISUALLY HIDDEN NATIVE CHECKBOX OVERLAY

In the menu layout, the native checkbox input element is placed in the same grid cell as the menu label. The dimensions of the checkbox are configured to fill its parent container, matching the dimensions of the menu label. The opacity is set to 0 so it is not visible, but remains accessible via pointers, keyboard, touch screens with haptics, and screen readers.

The checkbox is positioned in front of the menu label with z-index. The user can interact with the check box, but cannot interact with the label.

### NATIVE CHECKBOX STATE FUNCTIONALITY

The state of the checkbox input element determines the appearance of its subsequent sibling elements. The subsequent siblings of the checkbox are targeted using the subsequent sibling combinator.

©N. A. WEDDERBURN

| Checkbox State Psuedo Class | Sibling Display |
|---|---|
| **(none)** | • Custom check mark hidden – animate transform scale to 0<br>• Main Navigation links hidden – display none |
| **:checked** | • Custom check mark displayed – animate transform scale to 1<br>• Main Navigation links displayed – display flex |
| **:focus** | • Custom checkbox outlined |
| **:hover, :active, :focus** | • Custom checkbox highlighted – border color changed |

Table 7: Header Main menu checkbox states and resulting functionality

### Considered Implementations

#### DISPLAY NATIVE CHECKBOX

The native checkbox could be utilized by displaying the Header Main layout as a 4x2 grid with the checkbox and menu label in the top row, center cells:

| | | | |
|---|---|---|---|
| (null) | Checkbox | Menu Label | (null) |
| Menu Links | | | |

Table 8: Header Main layout as 4x2 grid with native checkbox.

A pseudo element on the Header Main element could span the center columns of the first row and display a box shadow, encapsulating the checkbox and label.

| | | | |
|---|---|---|---|
| (null) | Checkbox | Menu Label | (null) |
| Menu Links | | | |

Table 9: Header Main layout as 4x2 grid with native checkbox and first row drop shadow.

A custom checkbox allows for more control of the appearance and animations.

Also, some screen readers announce the label contents for every direct interaction with the label. The label contents are only announced once when interacting with the checkbox while the checkbox has focus. If the native checkbox does not overlay the label preventing interaction with the label, the user may inadvertently cause the label contents to repeat.

# Layout – Footer Main

©N. A. WEDDERBURN

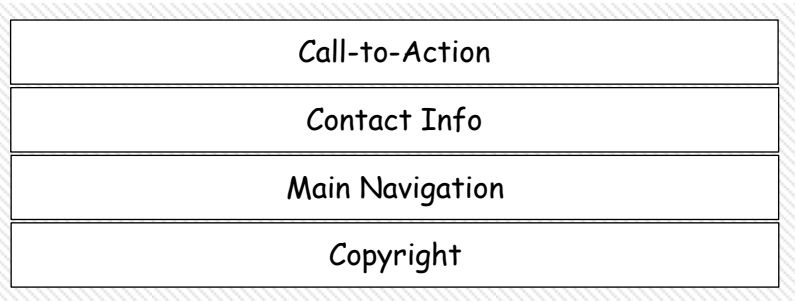



Figure 24: Footer Main layout, narrow viewport

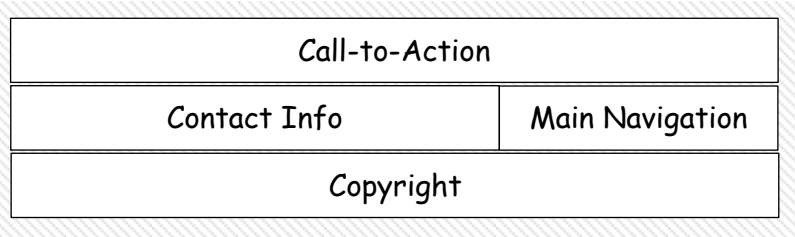



Figure 25: Footer Main layout, wide viewport

# Responsive Layout

For narrow screens, short screens, and print, the Footer Main has a flow layout. For wide/tall screens, a 5x3 grid layout is defined, with each column having equal width. The call-to-action and copyright elements each span 1 row and 5 columns. The contact information section spans 1 row and 3 columns, while the Main Navigation component spans 1 row and 2 columns.

| | | | | |
|---|---|---|---|---|
| Call-to-action | Call-to-action | Call-to-action | Call-to-action | Call-to-action |
| Contact | Contact | Contact | Main Navigation | Main Navigation |
| Copyright | Copyright | Copyright | Copyright | Copyright |

Table 10: Footer Main 5x3 grid layout

For screens, each footer component is configured with the same padding to maintain layout consistency.

# Text Appearance

For screen, the Footer Main is configured with a text color and base text size. All anchor elements in the Footer Main, including anchor components, are configured with the Footer Main text color.

# Component – Main Navigation

### *Layout*

The layout is configured as a column display. The configurations to change the flex direction to column are implemented.

# Page – Homepage

## Floated Page Title

The h1 page title is floated so its sibling elements wrap around it. A clear is added to the Featured Properties section which is first sibling element that should not wrap around the page title.

## Left Aligned Heading

The heading text in the Featured Properties section is left aligned for wide screens. The text alignment change also requires changing the alignment of the decorative pseudo element.

# Page – Office Information

## Broker Layout

In the DOM, each associate broker is configured as a section of the main element:

- Section Associate/Broker [section/.associate.broker]
    - Name [h4/.associate__name]
    - Avatar [img/.associate__avatar]
    - Biography [div/.associate__bio]

## Sales Associate Layout

In the DOM, a sales associate is configured as a figure element:

- Associate [figure/.associate]
    - Avatar [img/.associate__avatar]
    - Name [figcaption/.associate__name]

©N. A. WEDDERBURN

Note that there is no biography element. If necessary, another element structure can be used. Or, the name and biography elements can be added to the figcaption. The figcaption box can be removed from the layout via CSS using “display: contents”.

The list of sales associates is displayed as a responsive grid with repeating list items that contain the Associate components. Since the Associate component has a fixed width, the width custom property is overridden and set to “auto” so the component width is the width of its parent list item. Since the Associate component is not a grid item, its height must be set to 100% so it fills its parent list item’s dimensions.

# Page – Neighborhoods

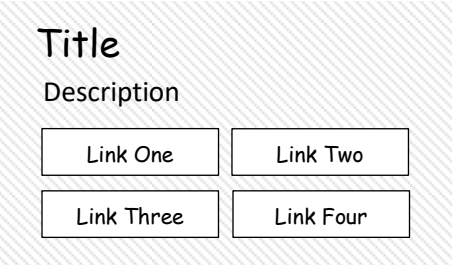



Figure 26: Neighborhoods page, small screen and print.

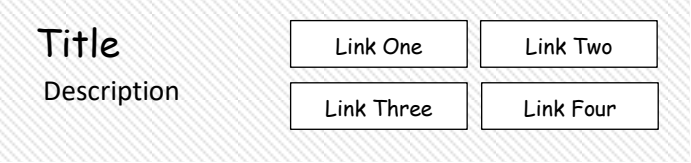



Figure 27: Neighborhoods page, large screen.

## Main Grid Layout

The main element is comprised of:

- Page title [h1]
- Neighborhoods Serviced section [section/.neighborhoods-serviced]
    - Neighborhoods Serviced title [h2/.neighborhoods-serviced__title]
    - Neighborhoods Serviced description [.neighborhoods-serviced__description]
    - Neighborhoods Serviced list [.neighborhoods-serviced__list]

The main element is displayed in a flow layout for small screens and print. It is displayed as a 2x2 grid on screens with large dimensions, both horizontal and vertical. In order to maintain semantics and create a grid where the grid items include the child elements of the Neighborhoods Serviced section, the section element’s box is replaced with its child elements by changing its display to “contents”.

©N. A. WEDDERBURN

The grid items are placed within defined grid areas.

| | |
|---|---|
| neighborhoods-title | neighborhoods-list |
| neighborhoods-description | neighborhoods-list |

Table 11: Neighborhoods page 2x2 grid layout for large screens

# List of Neighborhood Links

The list of neighborhood links is displayed as a responsive grid with repeating Anchor Primary components.

The Anchor Primary component is displayed as a single column grid. It contains a neighborhood name element and neighborhood URL element.

Figure 28: Neighborhood Anchor Primary component

The neighborhood URL element is configured to be hidden from screen readers with the aria-hidden attribute. The URL is also not displayed for print since anchors are configured to display their href attributes when printed.

## *Unsuccessful Implementations*

### HREF Displayed via Pseudo Element

The anchor href attribute can be retrieved and displayed as the content of the anchor element's pseudo element. Since the href attribute value is plain text, screen readers that use the Accessibility Tree as an alternate version of the DOM may announce the value. Also, the Anchor Primary component has an external link icon displayed in its :after pseudo element. So, the :before pseudo would be required to display the href attribute. With this configuration, the href attribute is displayed before the neighborhood name, requiring a reversal of the display order via CSS. Screen readers will continue to announce the elements in DOM order regardless of the order change via CSS.

©N. A. WEDDERBURN

APPENDIX

# References


1. A scalable naming convention for style-variables – Naming CSS custom properties with information on naming colors by contrast to the background color.
2. Building a color scheme – A foundational overview of how to establish a dynamic and configurable color scheme.
3. Text Wrapping & Inline Pseudo Elements – Adding an icon or indicator using a pseudo element so it appears before or after an element, such as an external link indicator.
4. Firefox Printing Issue for Grid CSS – Applying grid layout to HTML body element causes each element to print on its own page.
5. Printed Firefox page breaks with CSS grid element longer than one printed page – Stackoverflow post describing Firefox page breaks when printing a grid container spanning multiple pages.
6. CSS grid doesn't support word-wrap, overflow-wrap and hyphens properties – Github post describing a “defect” in CSS grid where words do not break when column widths are defined with fractional units. Workarounds are provided.
7. flex item overflows container due to long word even after using word-wrap – Stackoverflow post describing workarounds for flex containers with overflowing text even when overflow mitigation has been applied.
8. Pure CSS Hamburger Menu – Janessa Garrow’s implementation of a hamburger menu using a CSS checkbox and general sibling combinator.
9. Accessible hamburger buttons without JavaScript – Matt Simon’s augmentation of the CSS checkbox and general sibling combinator pure CSS hamburger menu. Extra anchor elements are implemented as keyboard focusable elements that can control the menu display.
10. Inclusively Hiding & Styling Checkboxes and Radio Buttons – Article by Sara Soueidan on inclusive methods for hiding input elements with a focus on haptics.
11. Forced Colors Mode strategies – Article by Matthias Zöchling detailing strategies for implementing forced colors.
12. Change SVG Color with Help from CSS Filter – Article by Paige Niedringhaus shows how applying a combination of filter functions can simulate color change on an SVG. Links to a StackOverflow thread and a CodePen that generates a filter based on a specified HEX color.
13. A Responsive Accessible Table – Adrian Roselli’s deep dive into constructing accessible HTML tables.


©N. A. WEDDERBURN

# Figures & Tables

Figure 1: The image on the left shows a short main section on a page with little content. The image on the right shows a lengthened main section on a page with little content. ..... 8

Figure 2: Anchor with child image that has height greater than the line height. ..... 12

Figure 3: Anchor with child image that has height greater than the line height, but the anchor is displayed as inline-grid to align the heights. ..... 12

Figure 4: Text anchor with inline-grid display not aligned with surrounding text. ..... 12

Figure 5: Anchor Primary component ..... 14

Figure 6: Anchor Secondary component ..... 15

Figure 7: Main Navigation ..... 16

Figure 8: Misaligned bordered Main Navigation element without text baseline alignment. ..... 17

Figure 9: Breadcrumb Navigation Component ..... 18

Figure 10: Figures Gallery grid, large screen width ..... 19

Figure 11: Figures Gallery grid, extra-large screen width ..... 19

Figure 12: Images displayed in a grid do not move to new page when printed with Firefox. ..... 20

Figure 13: Property component screen display ..... 21

Figure 14: Property container URL overlay stacking order ..... 22

Figure 15: Property component with URL overlay (green) and selectable text elements (red). ..... 23

Figure 16: Current Properties Matrix component ..... 24

Figure 17: Property component within Current Properties Matrix component on a large screen ..... 25

Figure 18: Sold Properties Matrix component ..... 26

Figure 19: Associate component ..... 26

Figure 20: Associate broker component ..... 26

Figure 21: Header Main layout ..... 27

Figure 22: Header Main, Main Navigation Component, wide screen. ..... 28

©N. A. WEDDERBURN

Figure 23: Header Main, Main Navigation Component, narrow screen ..... 28

Figure 24: Footer Main layout, narrow viewport ..... 33

Figure 25: Footer Main layout, wide viewport ..... 33

Figure 26: Neighborhoods page, small screen and print. ..... 35

Figure 27: Neighborhoods page, large screen. ..... 35

Figure 28: Neighborhood Anchor Primary component ..... 36

Table 1: Property component grid layout ..... 21

Table 2: Property component grid layout with preview and status is the same cell ..... 22

Table 3: Property component in Current Properties matrix - grid layout on large screen ..... 25

Table 4: Associate grid structure ..... 27

Table 5: Associate broker grid structure, wide/tall screen ..... 27

Table 6: Header Main collapsible Main Navigation grid layout ..... 30

Table 7: Header Main menu checkbox states and resulting functionality ..... 32

Table 8: Header Main layout as 4x2 grid with native checkbox. ..... 32

Table 9: Header Main layout as 4x2 grid with native checkbox and first row drop shadow ..... 32

Table 10: Footer Main 5x3 grid layout ..... 33

Table 11: Neighborhoods page 2x2 grid layout for large screens ..... 36

©N. A. WEDDERBURN